

Multiresolution Attributes for Hardware Tessellated Objects

Henry Schäfer, Magdalena Prus, Quirin Meyer, Jochen Süßmuth, and Marc Stamminger

Abstract—Hardware tessellation is one of the latest GPU features. Triangle or quad meshes are tessellated on-the-fly, where the tessellation level is chosen adaptively in a separate shader. The hardware tessellator only generates topology; attributes such as positions or texture coordinates of the newly generated vertices are determined in a domain shader. Typical applications of hardware tessellation are view dependent tessellation of parametric surfaces and displacement mapping. Often, the attributes for the newly generated vertices are stored in textures, which requires uv unwrapping, chartification, and atlas generation of the input mesh—a process that is time consuming and often requires manual intervention. In this paper, we present an alternative representation that directly stores optimized attribute values for typical hardware tessellation patterns and simply assigns these attributes to the generated vertices at render time. Using a multilevel fitting approach, the attribute values are optimized for several resolutions. Thereby, we require no parameterization, save memory by adapting the density of the samples to the content, and avoid discontinuities by construction. Our representation is optimally suited for displacement mapping: it automatically generates seamless, view-dependent displacement mapped models. The multilevel fitting approach generates better low-resolution displacement maps than simple downfiltering. By properly blending levels, we avoid artifacts such as popping or swimming surfaces. We also show other possible applications such as signal-optimized texturing or light baking. Our representation can be evaluated in a pixel shader, resulting in signal adaptive, parameterization-free texturing, comparable to PTex or Mesh Colors. Performance evaluation shows that our representation is on par with standard texture mapping and can be updated in real time, allowing for application such as interactive sculpting.

Index Terms—Signal-dependent storage, hardware tessellation, displacement mapping

1 INTRODUCTION

PROGRAMMABLE hardware tessellation has recently been established as a standard feature in the rendering pipeline. When activated, input geometry (which can be a triangle or a quad mesh) is automatically tessellated before being fed to the pipeline. More precisely, a tessellation control shader is called first to determine tessellation factors for the edges of a face as well as for its interior. Based on these values, the tessellator stage subdivides the input face using documented tessellation rules. The attributes of the newly generated vertices are then computed in an evaluation shader, that receives the barycentric coordinates for a vertex and determines its attributes such as its position or normal. From the resulting vertices, new small triangles are generated and sent further down the pipeline.

Hardware tessellation is designed to render parametric surfaces or for displacement mapping. In both of these applications, input geometry is subdivided according to the viewing distance, and the resulting mesh is displaced using the parametric description of the surface or its displacement map.

One issue with hardware tessellation is the question how to determine the attributes for the newly generated vertices. Depending on the attribute, three approaches are used in practice: First, attributes can simply be interpolated (e.g., texture coordinates). Second, some attributes can be determined procedurally from the barycentric coordinates (e.g., vertex positions and normals on a parametric surface). Third, the attributes can be read from a texture. In the latter case, an attribute is typically not fetched per vertex and then interpolated by the rasterizer, but it is directly read per pixel from the texture in the pixel shader. An important exception are displacement maps, from which values must be fetched per vertex.

Storing attributes in textures has a number of disadvantages: First, a parameterization for the objects is needed, which requires a segmentation of input geometry into charts, unwrapping the charts, and packing the unwrapped charts into a texture atlas. Second, it is difficult to adapt the texture resolution to a given signal, i.e., often large parts of a texture contain only little information. Finally, displacement mapping exhibits artifacts: when displacement values are read per vertex from a texture, they are interpolated. Undersampling results in well-visible artifacts at the silhouette of objects. When distance-based tessellation is used to reduce the tessellation of faces farther away from the observer, tessellation factors change, and the resulting mesh shows popping or *swimming* artifacts. Mipmapping does not solve this problem well. An example of texture related problems is depicted in Fig. 2. Another problem appears in conjunction with texture atlases: along texture seams slight discontinuities may appear due to slightly different values fetched from texture on both sides of a seam. While these discontinuities are hardly visible for

• H. Schäfer, M. Prus, J. Süßmuth, and M. Stamminger are with the University of Erlangen-Nuremberg, Cauerstr. 11, Erlangen 91058, Bavaria, Germany. E-mail: henry.schaefer@cs.fau.de.

• Q. Meyer is with Elektrotechnik Automotive GmbH, Am Wolfsmantel 46, Erlangen, Germany.

Manuscript received 31 May 2012; revised 18 Dec. 2012; accepted 7 Feb. 2013; published online 22 Feb. 2013.

Recommended for acceptance by M. Garland and R. Wang.

For information on obtaining reprints of this article, please send e-mail to: tvccg@computer.org, and reference IEEECS Log Number TVCGSI-2012-05-0098.

Digital Object Identifier no. 10.1109/TVCG.2013.44.

Authorized licensed use limited to: Lunds Universitetsbibliotek. Downloaded on April 10, 2025 at 16:06:56 UTC from IEEE Xplore. Restrictions apply.

1077-2626/13/\$31.00 © 2013 IEEE Published by the IEEE Computer Society

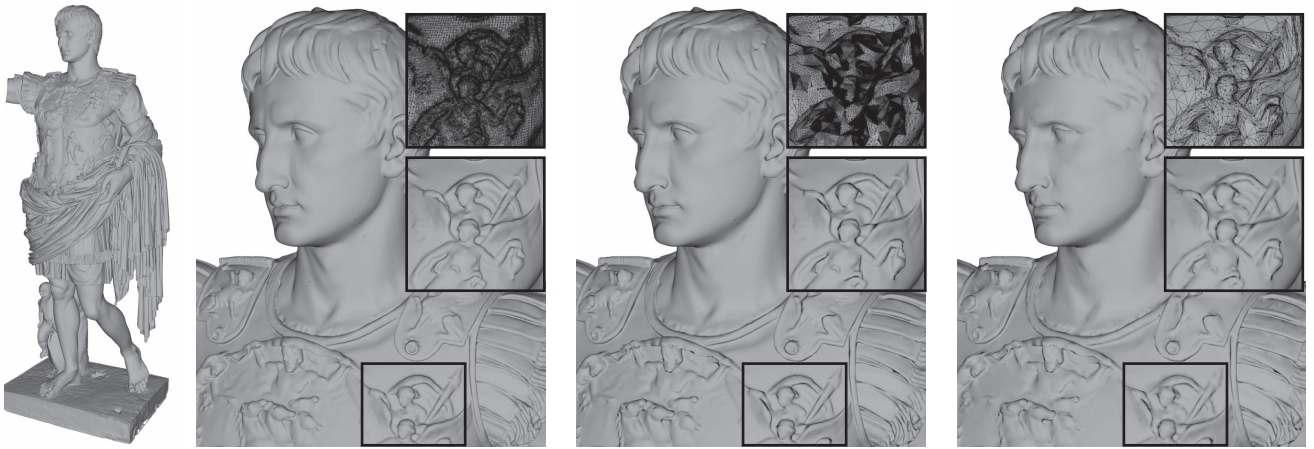


Fig. 1. Reconstruction of fine surface detail on a base mesh (100k triangle) generated from a 3D scan of an Augustus statue (height 2.2 m, 16M triangles) using our method for signal optimal displacement mapping. From left to right: rendering and wireframe closeups of scanned model (366 MB GPU memory, 13.1 ms rendering time), and our reconstructions with a maximum allowed error $\epsilon_{\max} = 1$ mm (90 MB, 5.14 ms) and $\epsilon_{\max} = 5$ mm (28 MB, 1.58 ms), respectively.

color textures, they result in cracks when using displacement maps (see Fig. 13(left)).

In this paper, we propose an alternative approach to generate attributes for tessellated meshes. For each face, we store the attribute values for the vertices generated by different tessellation factors in a simple list. Whenever the face is tessellated at runtime, we directly fetch the stored attribute values (see Fig. 2(right)). We further show how attribute values are optimized using a multiresolution fitting procedure. The resulting fitting error is used to control tessellation factors in a level-of-detail (LOD) fashion. By choosing the tessellation factor of each face such that the signal within the face is represented up to a user defined accuracy, we can easily adapt the required amount of data to the input signal.

Our approach can be used in any rendering application using tessellation, for example, for the rendering animated subdivision surfaces. It assigns attributes to newly generated vertices at high performance. Data resolution is signal optimized, and the signal is stored and reconstructed smoothly at multiple resolutions. Our method requires no parameterization and is fully automatic.

A particular useful application is to replace a detailed mesh by a coarse mesh and a displacement map, as shown in Fig. 1. Besides the dramatically reduced storage requirements for the index buffer, we save a significant amount of memory by storing displacement samples only where needed. At the same time, we can control the well-known undersampling and swimming artifacts, and completely avoid cracks along texture seams. By storing the attribute values directly in lists without making a detour via a texture, we reduce the number of stored samples and we assign attribute values directly at the position they were optimized for.

2 RELATED WORK

The ability to tessellate a coarse input mesh to generate massive amount of geometry directly in the graphics pipeline [1] is typically used to evaluate subdivision surfaces [2], [3], [4] and to add geometric detail via displacement mapping [5], [6]. Since all primitives in the input mesh are processed independently, it is important to create consistent transitions to prevent discontinuities, especially if different tessellation factors are used between adjacent patches.

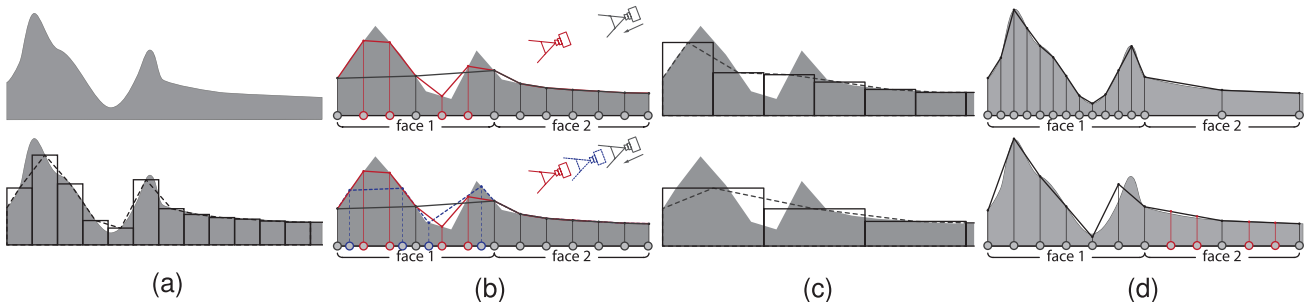


Fig. 2. A continuous signal ((a) top) is stored in a texture ((a) bottom). During rendering, the tessellation factors for each face are typically adapted to the camera distance. For integer tessellation factors, this results in popping artifacts as tessellation factors abruptly change at certain camera distances, and the displacement values are read at the newly generated vertices ((b) top). Fractional tessellation trades popping artifacts for swimming artifacts: The surface seems to flow ((b) bottom). Both problems appear due to undersampling the texture. Mipmapping displacement maps does not solve the problem well as shown in (c). Instead, we adapt the sample density to the input signal using a multiresolution fitting approach. Therefore, we tessellate the input geometry until a signal can be reconstructed by the generated vertices and store the signal as attributes ((d) top). At render time, we fetch the attributes if they are available in the attribute buffer or interpolate from our data structure (red vertices (d) bottom).

For displacement, problems appear when a texture atlas is used and interpolated texture values differ slightly on both sides of a texture seam. This affects either displacement normals or displacement values (or both). This leads to shading discontinuities or well perceivable cracks. Much research was spent on this issue recently; common approaches handle cracks by either precomputing transitions to stitch discontinuities [7], [8], [9] or creating a parametrization with consistent chart boundaries and orientation [10]; see Ni et al. [11] for a survey. Other methods prevent visible cracks by inserting triangle strips to fill holes [12] or by creating a consistent edge subdivision via incremental bisection of faces with a lower subdivision [13].

Other approaches avoid texture atlas problems using alternative signal representations. Ptex [14] uses per-face texturing by taking advantage of implicit parametrization of quad patches. Mesh Colors [15] virtually subdivide the mesh and use piecewise linear interpolation similar to vertex colors to reconstruct color signals. Kavan et al. [16] directly employ vertex colors by computing optimal attributes at vertices to resemble the input signal. Unfortunately, a pure vertex-based representation is suited only for low-frequency signals, since subtriangle detail cannot be represented that way. The hybrid signal adaptive method proposed by Schäfer et al. [17] solves this problem by combining vertex- and texture-based storage. They use optimal vertex colors for interpolation by default and textures only on faces, where vertex colors cannot reconstruct the input signal. Finally, they smoothly blend between both representations.

3 OVERVIEW

In the following, we assume that a coarse two-manifold mesh is given that is tessellated on-the-fly using hardware tessellation. One or multiple input signals are defined on this mesh, e.g., colors, displacements, normals, or lighting data. The signals can be defined by a texture, by a procedure (e.g., lighting or displacement of parametric surfaces), or, for displacement maps, by a fine mesh from which the displacements can be computed by ray casting. We propose to bake such signal(s) into vertex attributes of a tessellated mesh, ideally at multiple resolutions.

Our goal is to avoid textures, which suffer from the previously mentioned shortcomings. Instead, we store the attribute values for vertices generated by hardware tessellation in lists with each face. We do this for a number of different tessellation factors, resulting in *multiresolution attributes* (Section 4). Then, we compute attribute values that optimally represent the given signal for different tessellation factors, either by sampling the signal at the generated vertices or by fitting the attributes of the generated vertices to the input signal. By controlling tessellation factors per face using a bounded error, we obtain a signal-adaptive representation (Section 5). At render time, the attribute values for all newly generated vertices are fetched—or interpolated—from our data structure. We describe how the values can be extracted efficiently from our data structure and give details on filtering and mipmapping in Section 6.

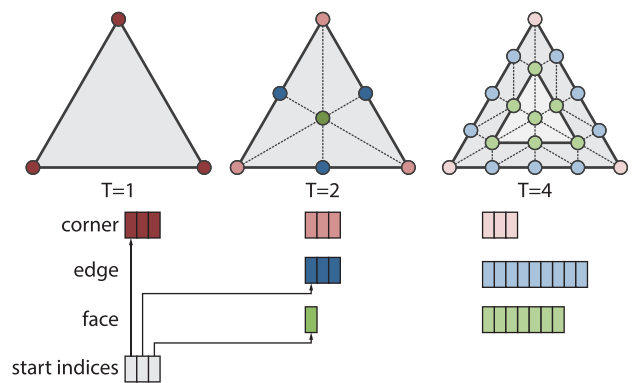


Fig. 3. Separate attribute storage for corner, edge, and face vertices in their corresponding arrays and a face descriptor with start indices to the arrays shown for the first three power of two tessellation levels of a triangle face. This storage scheme allows us to share attributes at edges of adjacent faces.

4 DATA LAYOUT

Similar to Mesh Colors [15] or Ptex [14], we store data per face. However, our subdivision scheme is guided by the fixed patterns of the hardware tessellator, i.e., we store all vertex attributes that correspond exactly to the vertices generated by the tessellation unit in a simple array.

To support adaptive tessellation, we generate attribute sets for different tessellation factors. For example, we store the signal for each power of two tessellation factor similar to texture mipmapping. Vertex attributes are stored separately for vertices corresponding to corner, edge, and face vertices as depicted in Fig. 3 exemplary for a triangle face. Thus, edges and corner vertices between adjacent faces share the same attributes resulting in a continuous signal along face boundaries.

We store all attribute data in a single attribute buffer and additionally for each face a descriptor containing lookup indices to the first attribute values of the face's corner, edge, and face vertices. For edge and corner vertices, we store attributes only once, and assign each of the adjacent faces a lookup index to the first edge vertex value. To incorporate the face orientation and opposite read directions of edges at adjacent faces, we use signed lookup indices to access edge attributes. During retrieval, we reconstruct the read direction of the edge based on the sign of its lookup index as described in Section 6.1.

For multiresolution attributes, we make sure to store data corresponding to one vertex, edge, or face for all of its levels one after another. This allows us to use only one index per list and to compute the offset to the next level on-the-fly. Optionally, we store for each corner vertex the maximum error of the vertices in the adjacent faces measuring the maximum difference between the sampled signal and the attributes at the generated vertices occurring at a particular level. This will later allow us to use signal-adaptive tessellation as discussed in Section 6.3.

After a face has been tessellated using tessellation factors for which we have stored attributes, we simply assign our data to the attributes of the generated vertices, and the interpolation will automatically be performed by the graphics hardware. For intermediate tessellation factors, we use filtering techniques as described in Section 6.

5 PREPROCESSING

In a preprocessing step, we convert all input signals, e.g., normals, colors, and displacement, that are defined on the surface of a base mesh into a representation such that the signal can later be reconstructed as vertex attributes of the tessellated base mesh. For each signal, we determine the tessellation factor that is necessary to reproduce the signal faithfully. This results in a *signal optimal* representation, as within each face, we only store as much data as needed to reconstruct the input signal up to a given accuracy.

First, we convert the input signal into a discrete set of sample points that lie on the base mesh as proposed by Schäfer et al. [17]: For each sample position s_j —this may be a texel midpoint, a mesh color, a stroke in an interactive sculpting or painting application, or a sample taken from a continuous function—let $\alpha_j, \beta_j, \gamma_j$ be the barycentric coordinates w.r.t. the corner vertices $v_{t,0}, v_{t,1}, v_{t,2}$ of a (tessellated) triangle t . Further, let $f(s_j)$ be the signal at the position s_j of the sample point.

Since we store the signal as per-vertex attributes of a (tessellated) mesh, the signal value $f^{\text{inter}}(s_j)$ at position s_j will later be a barycentric interpolation of the attribute values $f(v_{t,0}), f(v_{t,1}), f(v_{t,2})$ at the corner vertices of triangle t :

$$f^{\text{inter}}(s_j) = \alpha f(v_{t,0}) + \beta f(v_{t,1}) + \gamma f(v_{t,2}). \quad (1)$$

To find the *signal optimal* vertex attribute representation of the input signal, we need to estimate the function values f_i at the mesh's vertices v_i such that the average difference between the actual signal values $f(s_j)$ and the interpolated signal value $f^{\text{inter}}(s_j)$ is minimized. As shown by Schäfer et al. [17], a least-squares solution to this problem can be efficiently found by solving a sparse Laplace-like linear system. To avoid solving an under-constrained system, we make sure that each (subdivided) face contains at least three signal samples. Additionally, we generate samples at mesh boundaries to prevent extrapolation during the fitting process.

In real-time sculpting or painting applications, where a fitting step on the CPU would be too expensive, the signal values at vertices can alternatively be sampled from the input signal.

Next, we compute for each face t , whether the signal within the face is reconstructed accurately enough. To this end, we calculate the *maximum* deviation ε_t between a function sample $f(s_j)$ and the linear interpolation $f^{\text{inter}}(s_j)$ corresponding to the current tessellation factor:

$$\varepsilon_t = \max \| f^{\text{inter}}(s_j) - f(s_j) \| \quad \forall \quad s_j \in t. \quad (2)$$

If this error is below a user defined threshold ε_{max} , we say that no finer tessellation is necessary and consider the face to be *converged*.

For all faces that are not yet converged, we further subdivide by doubling the tessellation factor. This refinement produces new vertices and, thus, allows us to represent the input signal more faithfully as vertex attributes. We repeat the subdivision and fitting process until ε_t reaches the threshold or the maximum tessellation level is reached. As a result, we obtain a mipmap-like representation of the signal, where the maximum depth is chosen according to the signal.

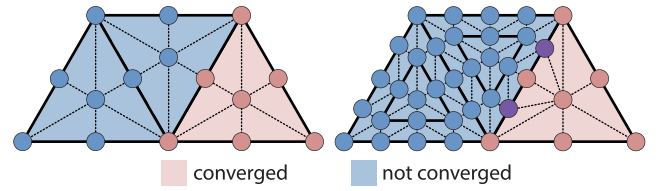


Fig. 4. Locking converged vertices during the proposed hierarchical fitting process. The pink faces have converged at level 1. Therefore, their vertices (pink) are locked when fitting subsequent levels. The purple vertices on the edge between the pink and the light blue face have been inserted to produce a crack-free mesh. We ensure that the signal within a converged face is not changed anymore: Therefore, the attribute values at the purple points are linearly interpolated between their neighbors on the edge. Finally, the purple vertices are locked as well.

For each mipmap level, we store the corresponding object space error ε_t . At render time (see Section 6), we use these error values to decide about the appropriate tessellation factor for a given viewing distance.

We avoid discontinuities between faces that require different tessellation factors, by setting the tessellation factor of each edge to the maximum tessellation factor of its adjacent faces. This ensures that the result of the tessellation is always a watertight mesh without T-junctions, thus, guaranteeing that the signal is continuous along edges. When computing the vertex attributes of the refined tessellation mesh, we need to take care that the signal within faces that have converged at an earlier level is not changed anymore, as this could push the interpolation error within these faces above the threshold ε_{max} again. Therefore, we *lock* all vertices that belong to a converged face, as shown in Fig. 4. Vertices on edges of locked faces introduced by higher tessellation factors of adjacent triangles are locked as well and the respective attributes are linearly interpolated along the edge. We handle locked vertices by treating them as fixed constraints within the system of linear equations, i.e., we eliminate the corresponding variables from the linear system. This reduces memory requirements and improves fitting speed, as usually only a fraction of the mesh's vertices has to be fitted at each level.

The proposed fitting scheme offers two advantages: First, our representation is signal optimized, as for each face, we only store the amount of data that is required to reconstruct the signal. A second advantage of our hierarchical fitting scheme is that the intermediate fitting results provide signal-optimized mipmaps. When fitting a model up to the desired accuracy, we always increase the tessellation factor of nonconverged face by a factor of two. Thus, by storing the intermediate fitting results, we obtain a power-of-two MIP-map representation that also approximates the input signal in a least-squares sense.

6 RENDERING

Graphics hardware provides two programmable stages for tessellation: the *hull shader* is executed per input face and controls tessellation unit by defining edge and face tessellation factors. Then, the mesh is tessellated and the *domain shader* is executed for each newly generated vertex. During domain shading, vertex attributes are assigned.

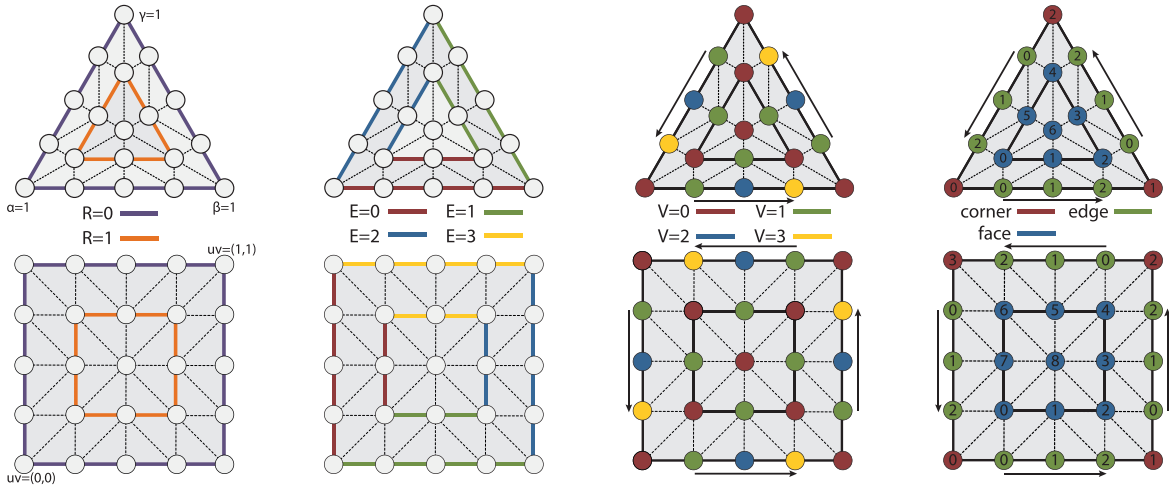


Fig. 5. Conversion from barycentric coordinates to unique indices for triangle (top) and quad (bottom) tessellation patterns. From left: ring, edge, vertex coordinates, and face-local unique indices for corner, edge, and face vertex attribute lookup.

In the following, we describe how attributes are assigned to tessellated meshes using our storage scheme. We start with the simplest case, where tessellation factors, set in the hull shader, exactly match factors for which we have data available in our attribute buffer. In the next step, we show how filtering methods are applied to support arbitrary tessellation factors. Additionally, we describe how adaptive tessellation is combined with our multiresolution storage scheme to achieve watertight meshes and smooth blending between stored levels. Finally, we show that for color signals our storage scheme can also be efficiently evaluated in a fragment program.

6.1 Attribute Assignment

To assign attributes in the domain shader, a lookup into our data structure is required. Unfortunately, the tessellation hardware does not provide a unique index for the generated vertices. Instead, only the barycentric coordinate w.r.t. to the input face is available. Hence, we have to derive a unique index from the barycentric coordinates to perform the lookup. In the following, we describe the index computation in detail for the triangle and quad tessellation pattern.

6.1.1 Triangle Index Computation

According to the OpenGL 4.2 tessellation specification, triangles are first subdivided into $\lfloor T_{\text{inner}}/2 \rfloor$ concentric rings, where T_{inner} is the inner tessellation factor set in the hull shader. Edges on an inner concentric ring R , where R is defined as in Fig. 5, are then subdivided into $T_{\text{inner}} - R \cdot 2$ edges. Finally, the outer edges are subdivided using the edge tessellation factors.

This tessellation scheme allows us to compute a unique index for each generated vertex on a face from barycentric coordinates (α, β, γ) in the domain shader. For a given barycentric coordinate, we determine the corresponding concentric ring R , the edge E on that ring, and finally the vertex index V on that edge as shown in Fig. 5(left) with

$$R = \text{round}\left(\frac{3}{2} T_{\text{inner}} \cdot b_{\min}\right), \quad (3)$$

where $b_{\min} = \min(\alpha, \beta, \gamma)$. The edge index E can be directly derived with

$$E = \begin{cases} 0, & \alpha = b_{\min} \\ 1, & \beta = b_{\min} \\ 2, & \gamma = b_{\min}. \end{cases} \quad (4)$$

We compute the vertex index by projecting the coordinate and its corresponding edge onto the outermost triangle in a barycentric manner. The vertex index then depends on the number of vertices on that edge. Obviously, we have to distinguish between vertices on the inner and outer edges of a triangle. Points inside the triangle always refer to the same tessellation factor T_{inner} , whereas points on the outermost concentric triangle refer to the edge tessellation factors $T_{\text{outer}}^0, T_{\text{outer}}^1, T_{\text{outer}}^2$. A general computation of the vertex index V is given by

$$V = \frac{b'_E}{1 - 3b_{\min}} (T - 2 \cdot R), \quad (5)$$

where $b' = [\gamma \ \alpha \ \beta]^T$ are the components of the barycentric coordinate, where the angle bisector intersects the edge

$$T = \begin{cases} T_{\text{outer}}^E, & R = 0 \\ T_{\text{inner}}, & \text{otherwise.} \end{cases} \quad (6)$$

The resulting ring, edge, vertex (REV) indices are then used to compute a linear index as shown in Fig. 5(bottom). For face vertices, the lookup index in the attribute buffer is computed by adding the face start index to the relative vertex index. We guarantee consistent edges between adjacent faces by storing shared edges only once. The look-up index for edges is thus a signed index indicating the reading order, where for positive indices V remains the same and for negative indices we use $V = T - V$.

6.1.2 Quad Index Computation

The index computation for quads follows the triangle index retrieval described above. First, we determine the local ring, edge, vertex (REV) indices for given barycentric coordinates (u, v) as shown in Fig. 5(right). Analogously to triangle tessellation, quads are also subdivided into

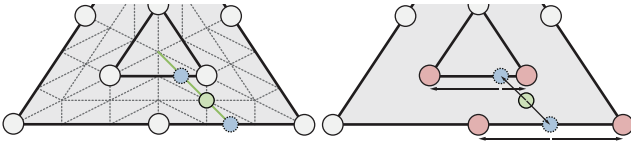


Fig. 6. Illustration of bilinear interpolation at the green vertex using data available in a coarser tessellation level (black). First, the vertex is projected to the closest inner and outer ring in the coarser level (blue). The four neighboring data points (red) as well as the interpolation weights are then easily computed using the ring, edge, vertex representation.

$\lfloor T_{\text{inner}}/2 \rfloor$ concentric rings. The ring index for quads is then defined as

$$R = \text{round}(T_{\text{inner}} \cdot b_{\min}), \quad (7)$$

where $b_{\min} = \min(u, v, 1 - u, 1 - v)$.

The edge index E is determined by the comparison of the two barycentric coordinates:

$$E = \begin{cases} 0, & u \geq v \ \& \ (1 - u) \leq v \\ 1, & u \geq v \ \& \ (1 - u) > v \\ 2, & u < v \ \& \ (1 - v) \leq u \\ 3, & u < v \ \& \ (1 - v) > u. \end{cases} \quad (8)$$

The vertex index follows from the ring and edge indices resulting in

$$V = b_{u,v} \cdot (T - 2 \cdot R), \quad (9)$$

where T is defined as in (6). $b_{u,v}$ depends on the edge index and is always the barycentric coordinate, which is not constant on that edge. For edges 0 and 2, this is the u coordinate, whereas for edge 1 and 3, $b_{u,v}$ corresponds to the v coordinate.

6.2 Filtering

In practice, arbitrary tessellation factors are desired. Therefore, we need to assign attributes to vertices generated in between stored attribute positions. In the following, we describe how nearest neighbor, bilinear, and trilinear filtering is performed using our storage scheme. Let T_{current} be the currently used tessellation factors for rendering and T_{data} be the factors for which we have data available in the attribute buffer. For both interpolation schemes, we use the ratio $q = T_{\text{data}}/T_{\text{current}}$ to find attributes in our data set.

6.2.1 Nearest Neighbor Filtering

Nearest neighbor filtering requires to find the nearest point v_n to the current vertex position v_p , for which we have an attribute value. First, we compute the *REV* representation of the current vertex (v_p). The nearest neighbor in our data set can then directly be computed from the *REV* representation by rounding the indices depending on the ratios between a faces attribute buffer tessellation factors and the rendering tessellation factors:

$$\begin{aligned} R_n &= \text{round}(q \cdot R_p), \\ E_n &= E_p, \\ V_n &= \text{round}(t \cdot V_p), \end{aligned} \quad (10)$$

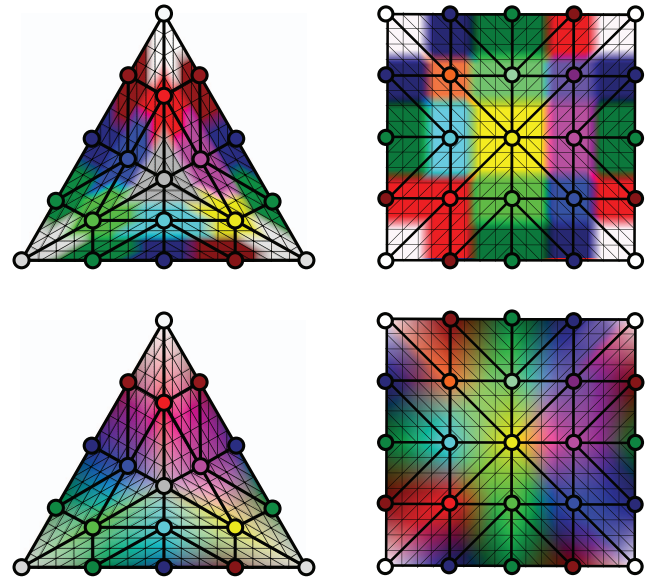


Fig. 7. Example of nearest neighbor (top) and bilinear attribute interpolation (bottom) for vertices in the current tessellation level using attributes available in a different level (black edges) for triangle (left) and quad (right) faces.

where t is the ratio of generated segments on the edge of the different tessellation levels $t = (T_{\text{data}} - 2 \cdot R_n) / (T_{\text{current}} - 2 \cdot R_p)$.

6.2.2 Bilinear Filtering

For bilinear filtering, we determine the attribute value from the four nearest neighbors for which we have data in the attribute buffer. Similar to nearest neighbor filtering, we start by computing the ring, edge, vertex representation of the current vertex. But instead of rounding, we compute fractional values for the ring and vertex. Then, we use simple clamping to the closest inner and outer ring and left and right vertex on these rings to determine the four neighbor indices. The final computation is then done easily with the fractional ring and vertex values as depicted in Fig. 6. Results of both filtering methods applied to triangle and quad faces are shown in Fig. 7.

6.2.3 Trilinear Filtering

To support multiple resolutions, we store optimal attribute values for all intermediate tessellation levels up to the maximum required tessellation. Since we store attributes for faces generated by using power-of-two tessellation factors, this is comparable to mipmapping. But in contrast to MIP-mapping, where levels are generated fine to coarse by averaging pixel values, we generate optimal attributes coarse to fine. For arbitrary tessellation factors, we interpolate the stored values from the bracketing power-of-two tessellation factors, resulting in a trilinear interpolation as known from texture MIP-mapping.

6.3 LOD Rendering

6.3.1 Signal-Adaptive Tessellation

Our signal-adaptive storage scheme allows us to adapt tessellation factors according to the underlying signal(s) during rendering. To this end, we store with every vertex

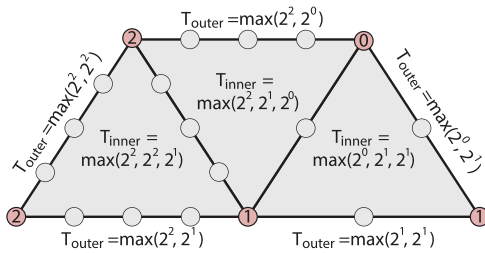


Fig. 8. During LOD rendering, the LOD is determined at corner vertices (red) using either an error- or distance metric. Inner and outer tessellation factors result from the corner vertices' LOD. Hence, the edge tessellation is a function of the corner vertices. This guarantees continuity and watertight transitions between adjacent faces.

of the base mesh a list of the maximum signal errors for each stored tessellation level in the surrounding faces. We use a user-defined quality parameter and assume that the vertex error scales with the reciprocal of the viewing distance. With a binary search on the vertex error list, we select the tessellation level that just meets the desired quality parameter.

The levels of detail computed at the corner vertices are then used to determine edge and inner tessellation factors as shown in Fig. 8. Finally, the faces are tessellated and attributes are assigned that meet the desired quality parameter.

6.3.2 Continuous LOD

As described before, only power-of-two tessellation factors will be chosen, resulting in popping as tessellation factors adaptively change with the camera distance. We achieve a smooth transition by using fractional tessellation factors. To this end, we have to compute a fractional tessellation factor for the vertices by interpolating tessellation factors from the vertex interpolation error list according to the quality threshold.

For most signals, this fractional tessellation delivers satisfying results. In case of displacement maps, however, sampling the signal in between the original samples leads to well-perceivable swimming artifacts—the resulting mesh seems to “flow” over the surface. This effect is also well known for displacement mapping with textures and smoothly adapted tessellation factors.

Therefore, we refrain from using fractional tessellation for index computation, but always use the coarsest power-of-two tessellation factor that meets our error criterion. We avoid popping by blending the displacement with the bilinear interpolation of the next coarser level. As a result, we get a smooth transition between the two levels, but only use displacements from their original sample position. The swimming and undersampling artifacts can, thus, be avoided.

6.4 Fragment-Based Evaluation

For color signals, we are only interested in the color attribute at the generated vertices. Hence, in contrast to displacement mapping, no modification of the vertex positions is required. Therefore, we can completely disable tessellation and instead evaluate attributes directly in a

fragment program. All we have to do is to compute for each fragment the barycentric coordinates w.r.t. fragments face.

Mesh Colors [15] upload each triangle individually for this purpose and assigns the barycentric coordinates to the separate vertices in an additional vertex attribute. As this does not allow to use shared vertices, the size of the vertex buffer is increased and additional memory for the barycentric attributes is needed.

A better solution is to use a simple geometry program that assigns barycentric coordinates to the vertices of the currently processed face. On graphics hardware, where geometry programs are not available, e.g., mobile devices, we can also compute the barycentric coordinate directly in the fragment program. Therefore, we assign the vertex position to an attribute in the vertex program. This is done for each vertex. Hence, the interpolated vertex position is then available in the fragment program. We attach the object's vertex and index buffer to the fragment program. This allows us to fetch the original vertex positions of a face and determine the barycentric coordinate from the interpolated vertex position. Then, we fetch data from the attribute buffer incorporating the filtering methods presented in Section 6.2. We compare the fragment-based evaluation with and without geometry program in the results section.

7 RESULTS

We implemented the proposed method for quad and triangle meshes. All timings shown in this section were measured at a screen resolution of $1,280 \times 720$ using an Nvidia GTX 680 and a Xeon E5620 CPU.

7.1 Preprocessing

In the following, we present the processing times to convert a full mesh with displacement signal to our representation. As input data, we use a terrain data set (see Fig. 9). The input data covers an area of 7.3 km^2 with a resolution of 1 m^2 for the displacement data stored in a texture. For the base mesh, we use a 70×70 quad plane. To give a quantitative and qualitative comparison for fitting and sampling at the generated vertices, we stop the algorithm after the first four tessellation levels (tessellation factors of 16 at maximum level). We compare the number of faces that are already converged and the maximum interpolation error achieved at this level. The results of the measurement in Table 1 show that a majority of the preprocessing time is spent on assigning existing signal samples to the generated triangles. This step involves costly point in triangle tests for each signal sample to determine to which tessellated face a sample belongs. This could be improved by performing the computations on the GPU. The comparison of fitting and sampling shows that fitting is more expensive, but converges faster and results in a lower remaining face error.

In sculpting or painting applications, usually only a fraction of the faces is affected by a brush, and fast updates of the data structures are required to fulfill real-time requirements. A direct attribute buffer update of 10 and 100 faces on the GPU at level three requires 2.72 and 5.47 ms, respectively, thus allowing real-time editing.

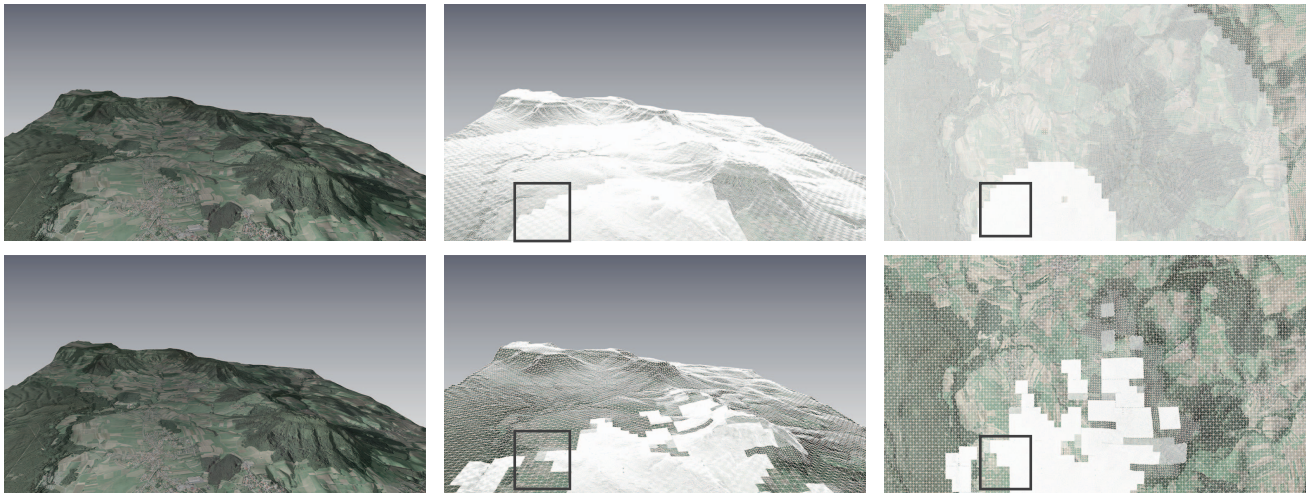


Fig. 9. Our data representation for terrain rendering using distance-dependent LOD (top row) and signal-dependent LOD (bottom row). From left to right: Standard rendering, wireframe overlay, and top view for the same camera to visualize subdivision levels. The view in the top row using standard distance-dependent LOD takes 0.63 ms to render, the same view in the lower row using our signal-dependent LOD metric takes 0.42 ms to render whereas displacement from texture takes 0.61 ms. The terrain data were extracted from “Geospatial Reference Data 2011” (Bavarian Administration for Surveying).

7.2 Filtering

We presented lookup and filtering for our storage scheme including nearest neighbor, bilinear, signal-dependent tessellation, and trilinear filtering that we employ for continuous LOD rendering. To give a performance comparison to standard textures, we use the terrain data set with a displacement signal as described before. For the filtering methods, we use standard distance-dependent tessellation, and let the tessellation factor increase reciprocal to the viewing distance. This increases tessellation of faces closer to the camera where surface detail is more important than on faces farther away. The results for signal-dependent and continuous LOD are presented in Fig. 9. In Fig. 10, we show difference images of the reconstruction using displacement from texture and our method to the original displacement. Details on rendering performance are given in Fig. 11.

TABLE 1
Terrain Preprocessing Comparing Fitting to Sampling the Displacement Signal at the Generated Vertices up to Mipmap Level Four

	fitting	sampling
assign samples to subdiv. triangles	23.41 s	24.27 s
fitting or sampling	5.72 s	0.53 s
compute face errors	0.21 s	0.29 s
converged faces	86.4 %	82.9 %
maximum remaining face error ε_t	0.014	0.018

The bottom row shows the maximum remaining face error at this level.

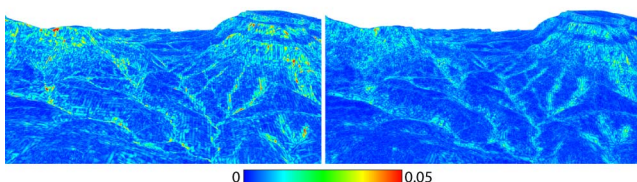


Fig. 10. Difference images measuring per fragment error between input signal and surface at tessellation level two for displacement from texture (left) and our method (right).

Although our method requires additional overhead (computing the index from barycentric coordinates and lookups to fetch data tessellation factors and start indices), it provides similar performance compared to displacement from texture. As expected, the rendering with bilinear interpolation is more expensive than nearest neighbor filtering, because three more lookups in the attribute buffer are required. Texture-based displacement cannot utilize the texture cache well for the used texture size and, thus, requires slightly more rendering time. For the continuous LOD, we require bilinear lookups in the bracketing power-of-two multiresolution attributes and also blending. Despite the increased rendering time, our method allows for continuous displacement mipmapping and LOD rendering without swimming or popping artifacts, which is—due to undersampling—not possible with displacement from texture.

To evaluate the reconstruction quality, we present a qualitative comparison for nearest neighbor and bilinear interpolation to standard texture mapping in Fig. 12. As shown in the close-up images, our method is able to reproduce the texture input signal similar to standard texture filtering without requiring an explicit parametrization. Other methods that do not require an explicit parametrization could be used for such signals as well.

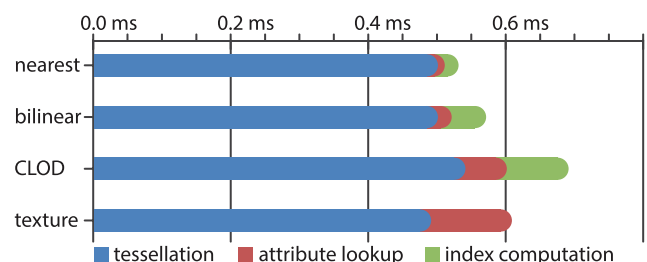


Fig. 11. Detail timings for index computation and buffer lookups needed by our method compared to displacement from texture for the terrain shown in Fig. 9 using distance-based tessellation.

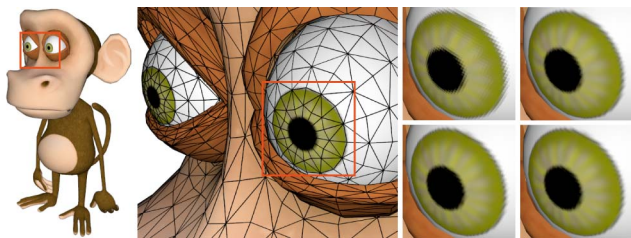


Fig. 12. Quality comparison for color signal; from left: rendering, closeup with wireframe overlay showing base geometry, closeup of the eye for texture (top), and our method (bottom) using the nearest neighbor and bilinear interpolation.

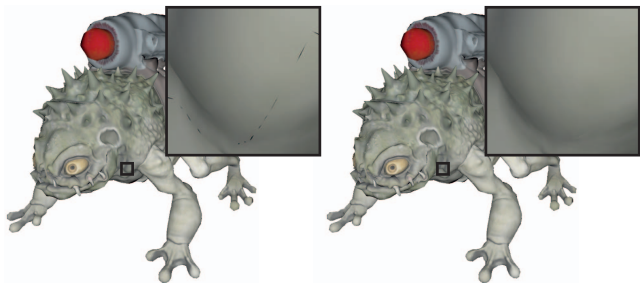


Fig. 13. Example for multisignal input data: the left image shows the original Monster Frog model displaced and colored using textures; the right image shows our rendering, where displacement, normal, and color were stored as attributes for vertices generated using hardware tessellation. The left closeup shows the cracks that are common along chart boundaries when using displacement textures. Our representation (right) does not suffer from such artifacts.

However, a pure vertex-based approach [16] would not be able to represent the signal on such a coarse mesh.

When texture atlases are used for displacement mapping, small differences in displacement values and normals on both sides of a seam can lead visible discontinuities in the displaced mesh. With our method, we avoid these problems; our edge data are always consistent as shown in Fig. 13 for the Monster Frog model (left: displacement from texture, right: our method). While the discontinuities can be avoided with methods described in Section 2, they do not prevent undersampling problems such as swimming or popping that appear using adaptive tessellation. Our method solves these problems by allowing to directly assign optimal attributes at the generated vertices.

With our method, it is also possible to store multiple signals. We applied multisignal storage to the Monster Frog model and, in addition to displacement values, converted normals and color data to our representation as shown in Fig. 13(right). When multisignal data are applied to the Monster Frog model, the rendering time increases to 2.58 ms for nearest neighbor and 2.71 ms for bilinear interpolation compared to 1.36 and 1.47 ms, respectively, for displacement only. The reason for the increased rendering time is that a distinct index has to be computed for each signal.

7.3 Per Fragment Evaluation

As described in Section 6, we can evaluate data for color signals directly in a fragment program without requiring hardware tessellation. This is possible, because for color data the positions of the generated vertices do not need to be modified as opposed to displacement. For a qualitative evaluation, we use a car model with baked ambient occlusion

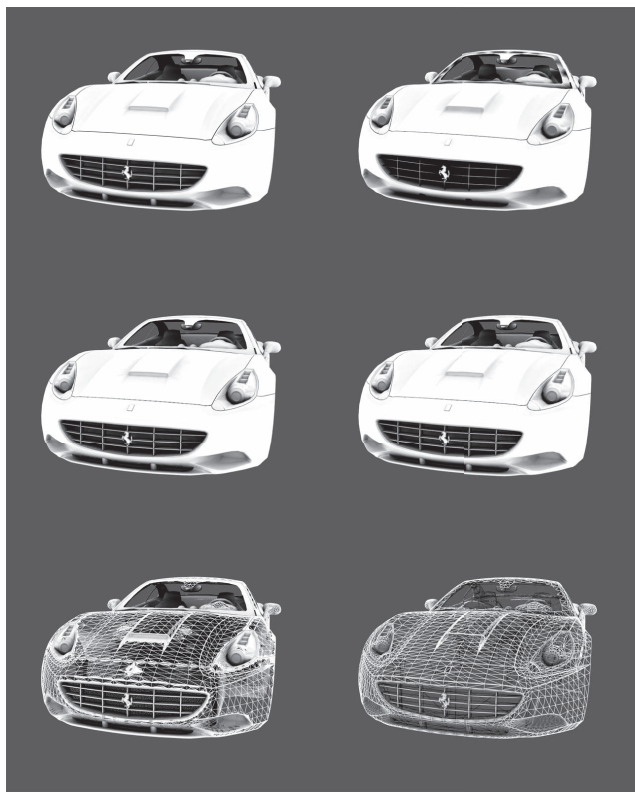


Fig. 14. Results for baked ambient occlusion using vertex baking (top right) and our method with evaluation of attributes in the domain shader (center left) and in a fragment program (center right) compared to the ray-traced reference (top left). The bottom row shows the wireframe model used for tessellation (left) and fragment-based evaluation (right).

shown in Fig. 14. We compare a ray-traced reference to baking only at the vertices against our method using tessellation and the per fragment attribute evaluation. Vertex-based baking (Fig. 14(top left)) shows clearly visible differences to the reference image at the windshield and radiator, since subtriangle information cannot be represented. The images in the middle row show that our method is able to reproduce the input signal well. For fragment-based evaluation, we presented a method with and without geometry program to get barycentric coordinates of the current fragment. When comparing the rendering times for tessellation (0.76 ms) and fragment-based evaluation (without geometry program 0.51 ms, with 0.44 ms) the latter provides a substantial speedup and is comparable to using textures (0.41 ms). The fragment-based rendering times show a minor win for using the geometry shader to generate barycentric coordinates on a face, because the method without geometry shader requires additional lookups to get the original vertex positions.

7.4 Memory Requirements

Our multiresolution storage scheme allows to adapt the sample density to the input signal. In the following, we present results for the memory requirements for the Augustus model shown in Fig. 1. The reference model is a 3D scan of the statue (height 2.2 m) consisting of 16.7M triangle faces. For the base mesh, we reduced the reference model to 100k faces using quadric edge collapse decimation [18]. In the preprocessing step, we use ray casting from the base to the reference mesh to get displacement values and detail mesh normals. Then, we converted the

TABLE 2

Comparison of Memory Requirements and Rendering Times with and without Trilinear Filtering Using Different Maximum Errors for the Augustus Statue Shown in Fig. 1

Augustus	Rendering default (ms)	Rendering mipmap (ms)	GPU Memory (MB)
Reference	13.1	—	366
$\epsilon_{\max} = 0.5$ mm	9.98	27.51	192
$\epsilon_{\max} = 1$ mm	5.14	14.24	90
$\epsilon_{\max} = 5$ mm	1.58	4.12	28

displacement signal using different maximum errors to measure the impact on rendering performance and memory requirements (Table 2). Note that the memory requirements also include all mipmap levels.

The achieved results show that by allowing minor differences per vertex we can greatly improve rendering performance and massively reduce memory requirements compared to the reference model. Another important benefit of our method for models with complicated surface detail like the toga of this statue is that we do not require an explicit parametrization.

8 LIMITATIONS AND FUTURE WORK

One major restriction on current hardware is that the maximum allowed tessellation factors on edges and faces is 64, which limits the signal frequency that can be reproduced by our approach. This is especially the case for models with a coarse base mesh and high-frequency signal. Performing a CPU subdivision on the mesh in a preprocessing step can solve this problem.

The hardware tessellator processes each face independently and does not expose information about neighboring faces. Therefore, we are currently restricted to filtering methods that do not filter over face borders. This could be solved by storing the adjacent faces of an edge in our data structure, similar to half-edge meshes. This will allow for improved filtering methods such as anisotropic filtering. Further, the hardware and shading languages do not provide a unique index to the generated vertices, so we have to compute an index from barycentric coordinates. If future hardware would expose an index, preferably as vector containing concentric ring, edge, and vertex indices, this would improve the rendering performance and make shaders significantly simpler.

A common problem with hardware tessellation and displaced surfaces is that no occlusion culling is performed at the geometry level. Hence, all patches are tessellated and evaluated even if they do not produce visible fragments. Since our method requires expensive buffer lookups in the hull and domain shader, it would greatly benefit from patch-based occlusion culling as presented by Niessner and Loop [19].

For fragment-based evaluation, we require barycentric coordinates that we have to generate with a geometry program or compute in the fragment program. If the hardware would expose the parameters used to interpolate the vertex attributes, the performance of all face-based texturing approaches like Ptex, Mesh Colors, and our method would improve.

In the future, we also plan to build an interactive sculpting and painting application based on our method, improve the preprocessing time by shifting the point in face tests to the GPU, and combine our displacement mapping with smooth Catmull-Clark subdivision.

9 CONCLUSION

We proposed a novel scheme for representing subface signals, which is optimized for meshes tessellated by graphics hardware. Storing the signal as vertex attributes of tessellated meshes offers two substantial advantages over classic textures: First, we do not require a texture atlas, which is generally difficult to compute automatically and introduces artifacts such as discontinuities between charts or problems when generating mipmaps. Second, our approach is fully signal adaptive: for each face, we store exactly the amount of samples that is necessary to accurately reconstruct the original signal. Therefore, for a wide range of signals, our approach is also much more memory efficient than standard textures. Further, with our method, vertex attributes are fetched at positions they were optimized for. This solves undersampling artifacts, such as swimming and popping, appearing for textures in combination with adaptive tessellation.

We further proposed a multiresolution fitting scheme that allows us to store data, such as colors, normals, and displacement in a signal-optimal manner. Typical texture operations such as bilinear interpolation or mipmapping can be performed directly on our data structure. Additionally, our approach does not require any modification of the input geometry, since it operates on a per face basis.

Finally, we showed that the proposed technique has a wide range of interesting applications, for example, LOD rendering, light baking, terrain rendering, or seamless displacement mapping.

ACKNOWLEDGMENTS

The authors would like to thank Bay Raitt of Valve Software for the Monster Frog model and the Bavarian Administration for Surveying for the terrain data. The Augustus data set is courtesy of the Collection of Classical Antiquities (University of Erlangen-Nuremberg). They also would like to thank the Blender Foundation for the YoFrankie project, which is the source of the monkey model.

REFERENCES

- [1] Microsoft Corporation, "Direct3D 11 Features: Tessellation Overview," [http://msdn.microsoft.com/en-us/library/ff476340\(v=vs.85\).aspx](http://msdn.microsoft.com/en-us/library/ff476340(v=vs.85).aspx), 2009.
- [2] M. Nießner, C. Loop, M. Meyer, and T. Rose, "Feature Adaptive GPU Rendering of Catmull-Clark Subdivision Surfaces," *ACM Trans. Graphics*, vol. 31, no. 1, article 6, 2012.
- [3] C. Loop and S. Schaefer, "Approximating Catmull-Clark Subdivision Surfaces with Bicubic Patches," *ACM Trans. Graphics*, vol. 27, no. 1, pp. 8:1-8:11, 2008.
- [4] C. Loop, S. Schaefer, T. Ni, and I. Castaño, "Approximating Subdivision Surfaces with Gregory Patches for Hardware Tessellation," *ACM Trans. Graphics*, vol. 28, pp. 151:1-151:9, 2009.
- [5] A. Valdetaro, G. Nunes, A. Raposo, B. Feijo, and R. de Toledo, "LOD Terrain Rendering by Local Parallel Processing on GPU," *Proc. Brazilian Symp. Games and Digital Entertainment '10*, pp. 181-188, 2010.

- [6] N. Tatarchuk, J. Barczak, and B. Bilodeau, "Programming for Real-Time Tessellation on GPU," AMD Whitepaper, 2010.
- [7] F. González and G. Patow, "Continuity Mapping for Multi-Chart Textures," *ACM Trans. Graphics*, vol. 28, pp. 109:1-109:8, 2009.
- [8] I. Castaño, "Next-Generation Rendering of Subdivision Surfaces," *Proc. ACM Talk at SIGGRAPH '08*, 2008.
- [9] P.V. Sander, Z. Wood, S.J. Gortler, J. Snyder, and H. Hoppe, "Multi-Chart Geometry Images," *Proc. Eurographics/ACM SIGGRAPH Symp. Geometry Processing (SGP '03)*, pp. 146-155, 2003.
- [10] N. Ray, V. Nivolières, S. Lefebvre, and B. Levy, "Invisible Seams," *Computer Graphics Forum*, vol. 29, no. 4, pp. 1489-1496, 2011.
- [11] T. Ni, I. Castaño, J. Peters, J. Mitchell, P. Schneider, and V. Verma, "Efficient Substitutes for Subdivision Surfaces," *Proc. ACM SIGGRAPH '09 Courses*, pp. 13:1-13:107, 2009.
- [12] M. Schwarz, M. Stuginski, and M. Stamminger, "GPU-Based Rendering of PN Triangle Meshes with Adaptive Tessellation," *Proc. Vision, Modeling, and Visualization (VMV '06)*, pp. 161-168, 2006.
- [13] H.R. Pakdel and F. Samavati, "Incremental Adaptive Loop Subdivision," *Proc. Int'l Conf. Computational Science and Its Applications (ICCSA '04)*, pp. 237-246, 2004.
- [14] B. Burley and D. Laceywell, "Ptex: Per-Face Texture Mapping for Production Rendering," *Computer Graphics Forum*, vol. 27, no. 4, pp. 1155-1164, 2008.
- [15] C. Yuksel, J. Keyser, and D.H. House, "Mesh Colors," *ACM Trans. Graphics*, vol. 29, no. 2, pp. 15:1-15:11, 2010.
- [16] L. Kavan, A. Bargteil, and P.-P. Sloan, "Least Squares Vertex Baking," *Computer Graphics Forum*, vol. 30, no. 4, pp. 1319-1326, 2011.
- [17] H. Schäfer, J. Süßmuth, C. Denk, and M. Stamminger, "Memory Efficient Light Baking," *Computer & Graphics*, vol. 36, no. 3, pp. 193-200, 2012.
- [18] M. Garland and P.S. Heckbert, "Surface Simplification Using Quadric Error Metrics," *Proc. ACM SIGGRAPH '97*, pp. 209-216, 1997.
- [19] M. Niessner and C. Loop, *Patch-Based Occlusion Culling for Hardware Tessellation*. Computer Graphics Int'l, 2012.



Henry Schäfer received the MSc (Diplom) degree in computer science from the University of Erlangen-Nuremberg, Germany, in 2009, and is currently working toward the PhD degree in the Computer Graphics Group at the University of Erlangen-Nuremberg. His research interests include realistic image synthesis, data-compression techniques and real-time rendering.



Magdalena Prus received the degree in computer science from the University of Erlangen-Nuremberg, Germany, in 2011. Since then she is a PhD researcher at the Computer Graphics Group in Erlangen. Her research interests are in the area of realistic rendering, level-of-detail representations, and geometric modeling.



rendering, data compression techniques, and parallel computing.

Quirin Meyer received both the diploma and doctoral degree from the University of Erlangen-Nuremberg in 2008 and 2012, respectively. He is a senior software engineer at Elektrobot Automotive GmbH, Germany, where his research is focused on real-time rendering of geographic maps. Prior to that, he was a research assistant at the University of Erlangen-Nuremberg and as a research-intern at Microsoft Research. His research includes real-time rendering, data compression techniques, and parallel computing.



nonrigid registration of real-time 3D scanner data.

Jochen Süßmuth received the diploma and the doctoral degree in 2005 and 2012, respectively, from the University of Erlangen-Nuremberg. He is currently a freelance consultant specializing in novel 3D acquisition systems. Prior to that he worked as a postdoctoral researcher and lecturer at the University of Erlangen-Nuremberg. His research interests include 3D scanning, surface reconstruction from point clouds as well as



in Munich. In 2012, he was elected into the Eurographics Executive committee. Since 2012, he has been also leading the DFG-funded research training group "Heterogeneous Image System."

Marc Stamminger is a professor for computer graphics at the University of Erlangen-Nuremberg, from which he also received a diploma in computer science as well as a doctoral degree in computer graphics. His research interest includes real-time computer graphics and global illumination. He served as a member of the program committees of all major computer graphics conferences, and was program co-chair of Eurographics 2009

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.**